

Kafka — 토픽 · 메시지 · 컨슈머 그룹

6주차의 메시지 브로커입니다. 6-2 의 compose(apache/kafka:3.7.0, 컨테이너 이름 kafka)로 띄운 상태를 기준으로 합니다. 명령은 모두 컨테이너 안의 /opt/kafka/bin 에 있습니다.

준비 완료 확인

```
docker logs kafka 2>&1 | grep "Kafka Server started"
```

bash

```
[2026-10-02 15:47:34,385] INFO [KafkaRaftServer nodeId=1] Kafka Server started (kafka.server.KafkaRaftServer)
```

토픽

```
docker exec kafka /opt/kafka/bin/kafka-topics.sh --bootstrap-server localhost:9092 \
  --create --topic orders --partitions 3 --replication-factor 1
```

bash

```
docker exec kafka /opt/kafka/bin/kafka-topics.sh --bootstrap-server localhost:9092 --list
docker exec kafka /opt/kafka/bin/kafka-topics.sh --bootstrap-server localhost:9092 --describe
--topic orders
```

```
Created topic orders.
```

text

```
orders
Topic: orders    TopicId: lJpPcjvTkm2YHDHqRM7hA PartitionCount: 3      ReplicationFactor: 1
Configs:
  Topic: orders  Partition: 0    Leader: 1      Replicas: 1    Isr: 1
  Topic: orders  Partition: 1    Leader: 1      Replicas: 1    Isr: 1
  Topic: orders  Partition: 2    Leader: 1      Replicas: 1    Isr: 1
```

명령	하는 일	확인한 결과
kafka-topics.sh ... --alter --topic orders --partitions 4	파티션 늘리기	출력 없이 성공
kafka-topics.sh ... --alter --topic orders --partitions 2	파티션 줄이기 — 안 됨	「2 would not be an increase」 오류
kafka-configs.sh ... --alter --entity-type topics --entity-name orders --add-config retention.ms=3600000	보존 기간 1시간	Completed updating config for topic orders.
kafka-configs.sh ... --describe --entity-type topics --entity-name orders	바꾼 설정 보기	retention.ms=3600000
kafka-topics.sh ... --delete --topic orders	토픽 지우기	목록에서 사라짐

메시지 보내고 받기 (키 포함)

```
printf 'u1:{"id":1}\nu2:{"id":2}\nu1:{"id":3}\n' | docker exec -i kafka /opt/kafka/bin/kaf bash
ka-console-producer.sh \
  --bootstrap-server localhost:9092 --topic orders --property parse.key=true --property key.se
parator=:

docker exec kafka /opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server localhost:9092 \
  --topic orders --from-beginning --max-messages 3 --property print.key=true --property print.
partition=true
```

```
Partition:0      u1      {"id":1}
Partition:0      u1      {"id":3}
Partition:2      u2      {"id":2}
Processed a total of 3 messages
```

text

같은 키(u1)는 같은 파티션(0)으로 갑니다. 그래서 키가 같은 메시지끼리는 순서가 지켜집니다.

컨슈머 그룹과 오프셋

```
docker exec kafka /opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server localhost:90 bash
92 \
  --topic orders --group g1 --from-beginning --max-messages 3

docker exec kafka /opt/kafka/bin/kafka-consumer-groups.sh --bootstrap-server localhost:9092 --
list
docker exec kafka /opt/kafka/bin/kafka-consumer-groups.sh --bootstrap-server localhost:9092 --
describe --group g1
```

```
console-consumer-75738
g1

Consumer group 'g1' has no active members.
```

GROUP	TOPIC	PARTITION	CURRENT-OFFSET	LOG-END-OFFSET	LAG
g1	orders	0	2	2	0
g1	orders	1	0	0	0
g1	orders	2	1	1	0
g1	orders	3	0	0	0

text

LAG 가 0 이면 다 읽은 것입니다. 처음부터 다시 읽게 하려면 오프셋을 되돌립니다. --dry-run 으로 먼저 보고, --execute 로 실제로 바꿉니다. 되돌린 뒤 LAG 가 2:1 로 다시 생깁니다(실측).

```
docker exec kafka /opt/kafka/bin/kafka-consumer-groups.sh --bootstrap-server localhost:909 bash
2 \
  --group g1 --topic orders --reset-offsets --to-earliest --execute
```

토픽에 몇 개 쌓였나 — 오프셋 보기

```
docker exec kafka /opt/kafka/bin/kafka-get-offsets.sh --bootstrap-server localhost:9092 -- bash
topic orders-test
```

「토픽:파티션:마지막 오프셋」입니다. 메시지 2개를 보낸 뒤라 2 입니다.

막힐 때 (실측)

- `kafka-run-class.sh kafka.tools.GetOffsetShell` → 「Could not find or load main class kafka.tools.GetOffsetShell」.
Kafka 3.7 에서 옮겨졌습니다. 위의 `kafka-get-offsets.sh` 를 쓰거나, 클래스 이름을 `org.apache.kafka.tools.GetOffsetShell` 로 바꿉니다(둘 다 `orders-test:0:2` 로 같은 결과).
- 같은 이름으로 다시 만들면 「Topic 'orders' already exists」 — 이미 있는 것입니다. `--list` 로 확인합니다.
- 파티션은 늘릴 수만 있고 줄일 수 없습니다. 줄이려면 토픽을 지우고 다시 만듭니다(메시지도 사라짐).
- `--group` 없이 읽으면 `console-consumer`-숫자 같은 그룹이 자동으로 생깁니다. 그룹 목록에 모르는 이름이 보이면 이것입니다.
- 기동 직후 `Connection refused` 는 브로커가 아직 준비 중인 것입니다. 「Kafka Server started」 줄을 보고 칩니다.

데이터 엔지니어링 & RAG 파이프라인 · 부록 — 서버 주소(SERVER_IP)·계정(devuser)·비밀번호(CHANGE_ME...)는 자리표시입니다. 본인 환경의 값으로 바꿔 쓰세요.